Beadandó feladat dokumentáció

Készítette: Hevesi Georgina, Y2V0WV

e-mail: hvsbc@student.elte.hu

Feladat:

Készítsünk programot, amellyel a következő játékot játszhatjuk.

Adott egy $n \times n$ elemből álló játékpálya, amelyben Maci Lacival kell piknikkosarakra vadásznunk. A játékpályán az egyszerű mezők mellett elhelyezkednek akadályok (pl. fa), valamint piknikkosarak. A játék célja, hogy a piknikkosarakat minél gyorsabban begyűjtsük.

Az erdőben vadőrök is járőröznek, akik adott időközönként lépnek egy mezőt (vízszintesen, vagy függőlegesen). A járőrözés során egy megadott irányba haladnak egészen addig, amíg akadályba (vagy az erdő szélébe) nem ütköznek, ekkor megfordulnak, és visszafelé haladnak (tehát folyamatosan egy vonalban járőröznek). A vadőr járőrözés közben a vele szomszédos mezőket látja (átlósan is, azaz egy 3×3-as négyzetet). A játékos kezdetben a bal felső sarokban helyezkedik el, és vízszintesen, illetve függőlegesen mozoghat (egyesével) a pályán, a piknikkosárra való rálépéssel pedig felveheti azt. Ha Maci Lacit meglátja valamelyik vadőr, akkor a játékos veszít.

A pályák méretét, illetve felépítését (piknikkosarak, akadályok, vadőrök kezdőpozíciója) tároljuk fájlban. A program legalább 3 különböző méretű pályát tartalmazzon.

A program biztosítson lehetőséget új játék kezdésére a pálya kiválasztásával, valamint játék szüneteltetésére (ekkor nem telik az idő, és nem léphet a játékos). Ismerje fel, ha vége a játéknak, és jelezze, győzött, vagy veszített a játékos. A program játék közben folyamatosan jelezze ki a játékidőt, valamint a megszerzett piknikkosarak számát.

Elemzés:

- A játékban alapvetően 3 féle pályaméret ajánlott: a 8*8-as, a 10*10-es és a 15*15, de az alkalmazás bármely méret betöltésére képes egyszerű szöveges fájlból.

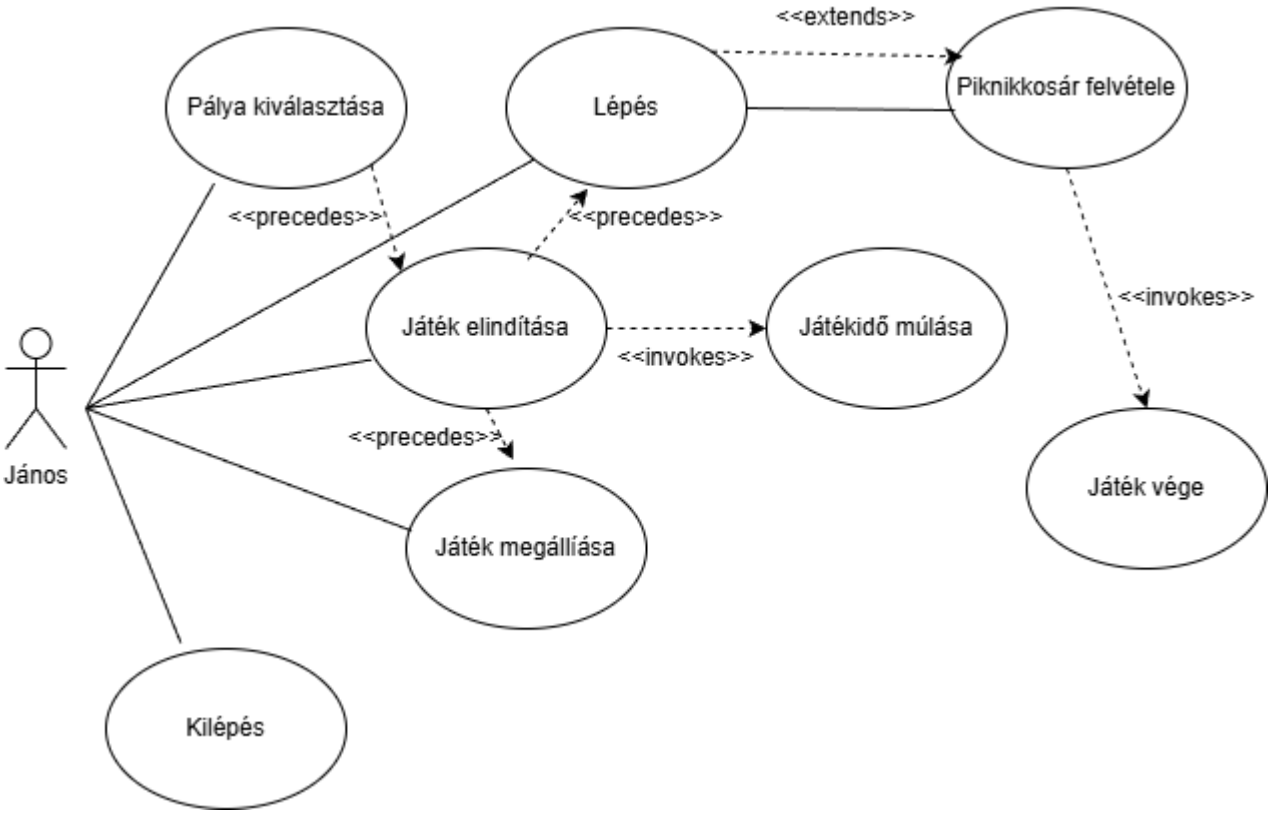
- A feladatot egyablakos asztali alkalmazásként .NET Avalonia grafikus felülettel valósítjuk meg.

- Az ablakban elhelyezünk egy menüt a következő menüpontokkal: **Game** (Játék betöltése, Játék megállítása/folytatása), **Rules.** Az ablak alján megjelenítünk egy státuszsort, amely a már összegyűjtött kosarak számát, illetve az eltelt időt jelzi.

- A játéktáblát egy n*n-es képekből álló rács reprezentálja. A nyilak megnyomására a karakter lép a pályán az adott irányban, ennek megfelelően frissül a megjelenítés is.

- A játék automatikusan feldob egy dialógusablakot, amikor vége a játéknak (nyertünk/vesztettünk). Szintén dialógusablakokkal végezzük el a betöltést vagy annak sikertelenségét, és a szabályok megjelenítését.

A felhasználói esetek az alábbi ábrán láthatóak:



Tervezés:

A programot MVVM architektúrában valósítjuk meg, ennek megfelelően **View, Model, ViewModel** és **Persistence** névtereket valósítunk meg az alkalmazáson belül. A program környezetét az alkalmazás osztály (**App)** végzi, amely példányosítja a modellt, a nézetmodell és a nézetet, biztosítja a kommunikációt, valamint felügyeli az adatkezelést. A program szerkezetét két projektre osztjuk implementációs megfontolásból: a Persistence és Model csomagok a program felületfüggetlen osztálykönyvtár projektjében, míg a ViewModel és View csomagok .NET Avalonia projektjeinek platformfüggetlen osztálykönyvtárában és Windows rendszerre szánt végrehajtahtó projektjében találhatóak.

Az adatkezelés feladata a játéktáblával kapcsolatos információk tárolása, valamint a betöltés biztosítása.

GameBoard osztály egy érvényes játéktáblát biztosít, ahol minden mezőre ismert az értéke, mellyel a játék kezdetekor és futása közben generált értékeket tároljuk. A tábla mérete példányosításakor meghatározott. A tábla lehetőséget a mezőértékek lekérdezésére, beállítására.

IDataAccess interfész lehetőséget ad a tábla betöltésére (**LoadAsync**), ezt hatékonysági okokból aszinkron módon valósítjuk meg. **FileDataAccess** osztály valósítja meg, a fájlkezelés során fellépő hibákat a **GameFileLoadInFailed** kivétel jelzi.

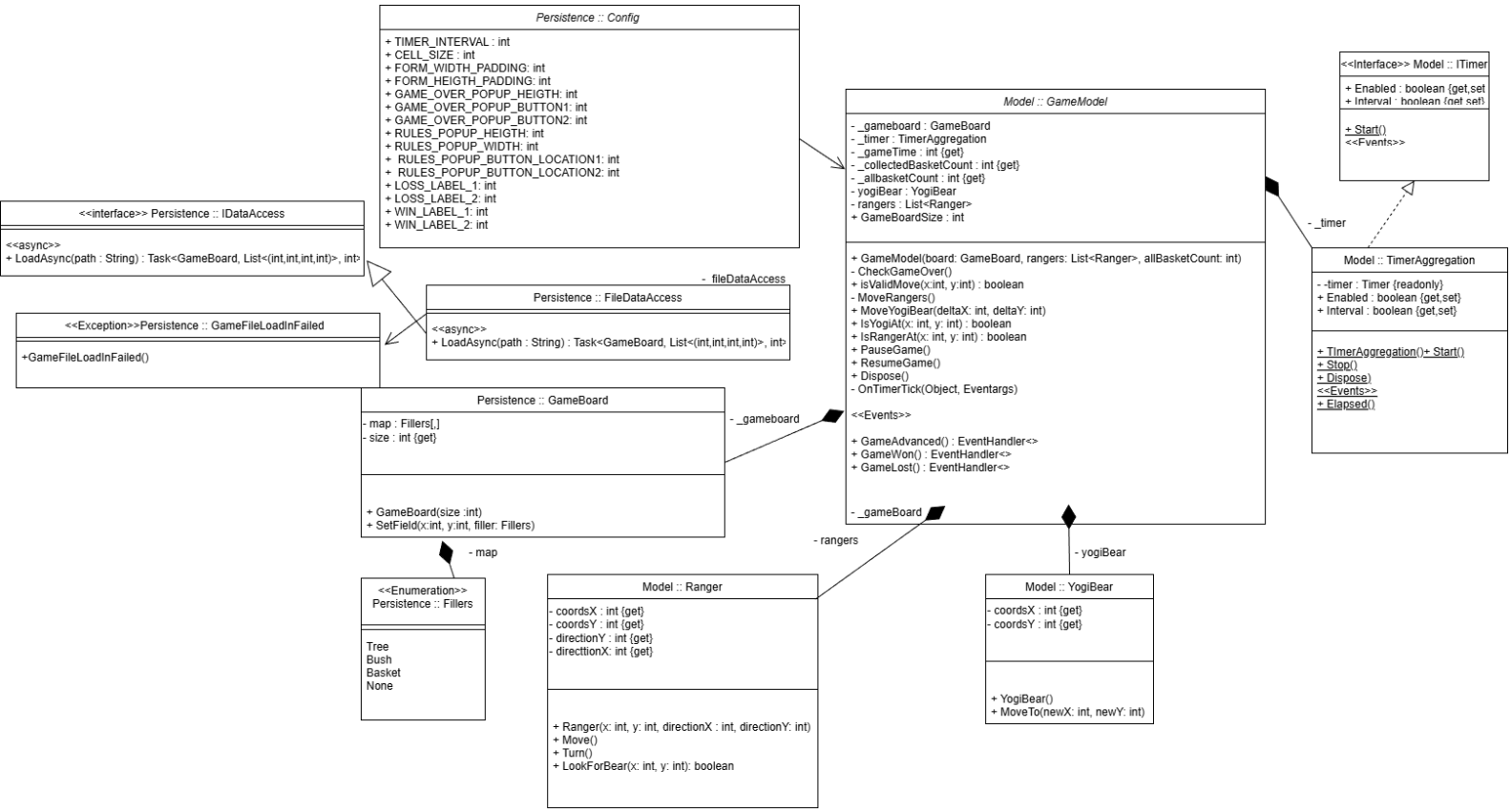
A betöltendő fájl első sora megadja a tábla méretét, a kosarak számát és a vadőrök számát. A fájl többi része izomorf leképezése a játéktáblának, azaz összesen méret + vadőrök száma sor következik, előbb a 4 adatból álló vadőr sorok, majd a tábla n hosszú sorai. A mezők értékei 0-3 értékeket vehetik fel.

Modell:

A modell lényegi részét a **GameModel** osztály valósítja meg amely szabályozza a tábla tevékenységeit, valamint a játék egyéb paramétereit. A típus lehetőséget ad a játék folytatására, megállítására (**PauseGame**, **ResumeGame**), valamint lépésre (**MoveRangers**, **MoveYogiBear**).

A játékidő kezelését egy időzítő végzi (**_timer**), amelyet megállíthatunk majd újra elindíthatunk. A játékállapot változásáról a **GameAdvanced** esemény tájékoztat.

Az alábbi ábrán látható a **YogiBearCore** projekt osztálydiagramja:



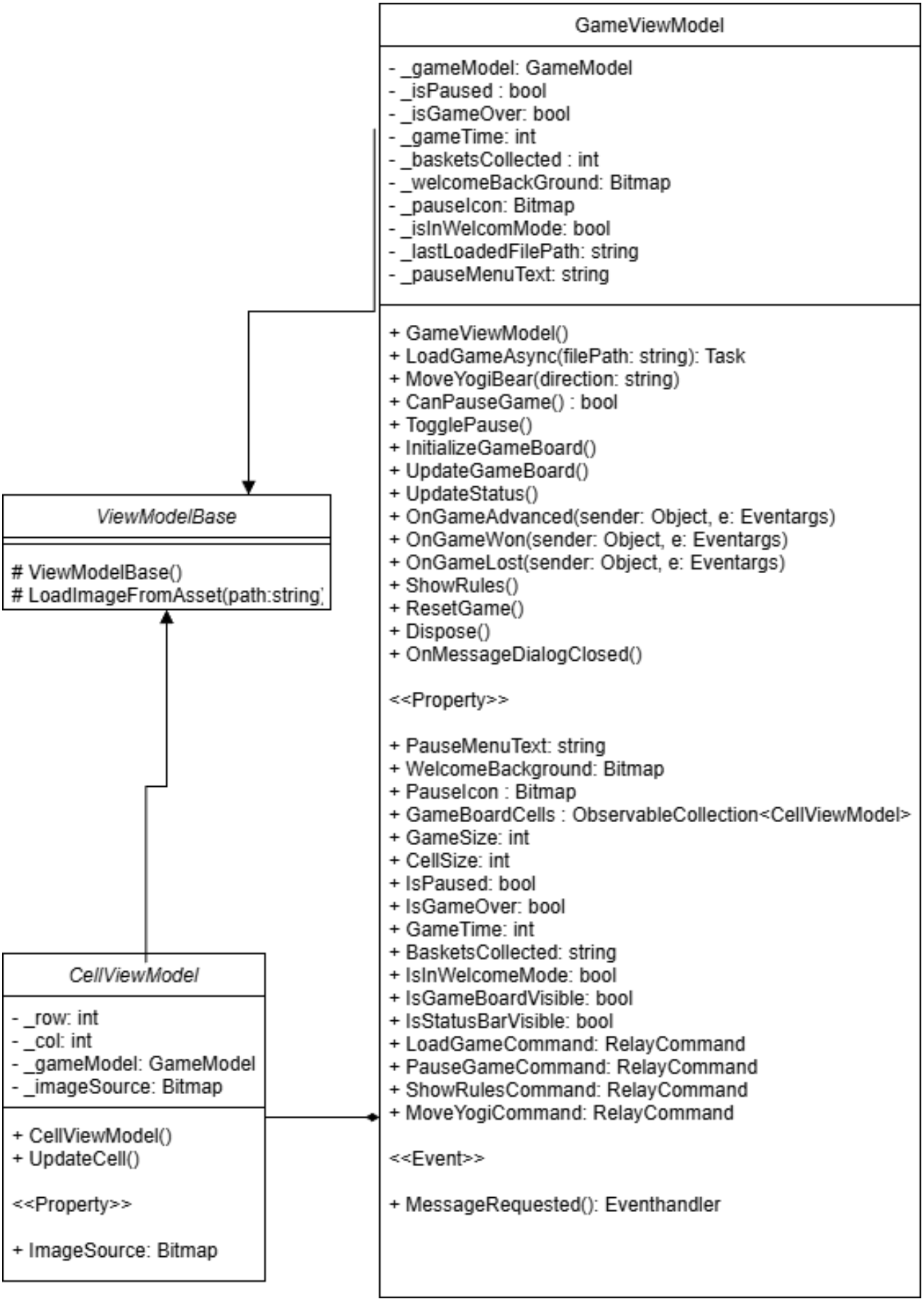
Nézetmodell:

A nézetmodell megvalósításához felhasználjuk az általános utasítás (**RelayCommand**) az MVVM Toolkit csomagból, valamint egy ős változásjelző (**ViewModelBase**) osztályt, mely implementálja az **ObservableObject** osztályt és biztosít egy közös képbetöltő metódus (**LoadInFromFilePath**) a nézetmodelleknek.

A nézetmodell feladatait a **GameViewMode**l osztály látja el, amely parancsokat biztosít az új játék kezdéséhez, a játék betöltéséhez. A parancsokhoz eseményeket kötünk, amelyek a parancs lefutását jelzik a vezérlőnek (App). A nézetmodell tárolja a modell egy hivatkozását **(_gameModel**), de csupán információkat kér le tőle, direkt nem avatkozik bele a játék futtatásába.

A játékmező számára egy külön nézetmodellt biztosítunk (**CellViewModel**), ami tudja magát frissíteni a _gameModel alapján. A mezőket egy felügyelt gyűjteményben (**GameBoardCells**) helyezzük el a GameViewModelben. Tartalmazza továbbá a réteg azokat az eseményeket, amelyekkel dialógusablakok megjelenítését tudja előidézni az Appban. A játéktábla generálását (**InitializeGameBoard**), illetve a játékértékek megjelenítését (**UpdateStatus**) külön metódusok végzik.

A nézetmodell osztálydiagramja:



Nézet:

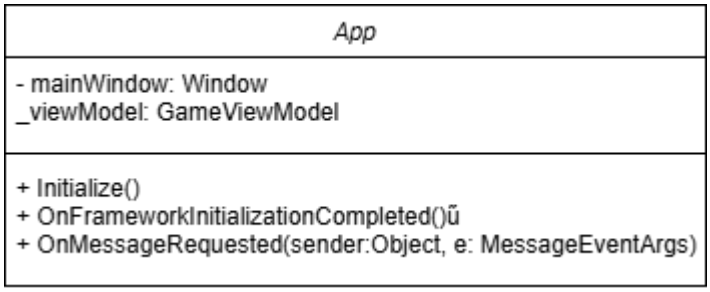
A nézet csak egy képernyőt tartalmaz, a **MainWindow** osztályt. A nézet egy rácsban tárolja a játéktáblát, a menüt és a státuszsort, amelyet egy panel zár egységbe a szebb megjelenítés érdekében. A játéktábla egy **ItemsControl** vezérlő, ahol dinamikusan felépítünk egy rácsot (**UniformGrid**), amely képekből áll. Minden adatot adatkötéssel kapcsolunk a nézetmodellhez.

A felületen létrehozzuk a megfelelő menüpontokat, illetve a státuszsort.

Környezet:

Az App osztály feladata az egyes réteget példányosítása (**OnFrameworkInitializationCompleted**), összekötése, a nézetmodell valamint a modell eseményeinek lekezelése, és ezáltal a játék, az adatkezelés és a nézet szabályozása.

A fájlnév bekérését, illetve a játék végét és a szabályok megjelenítését követő üzenetek megjelenítését MessageBox.Avalonia NuGet csomag segítségével itt végezzük.



Tesztelés:

A modell funkcionalitása egységtesztek segítségével lett ellenőrizve a **GameTest** osztályban. A modell időzítőjét egy **ITimer** interfészből származtatjuk le, így azt a tesztprojektben egy **MockTimer** megvalósítással mockolhatjuk, és az időzítőt explicit is triggerelhetjük (**RaiseElapsed**).

Az alábbi tesztek kerültek megvalósításra:

- Vadőrök mozgatása, fordulása
- Új játék felállítása
- Maci Laci mozgatása
- Kosarak felvétele
- Játék végi feltételek teljesülése
- Megállítás és folytatás funkciói